

Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №1

По дисциплине: «Модели решения задач в интеллектуальных системах»

Тема: «Реализация многослойного персептрона для решения задачи «исключающее ИЛИ»

Выполнила:
Студент 4 курса
Группы ИИ-26
Карпович К.А.
Проверил:
Никитин С.К.

Цель работы: Реализовать **многослойный** персептрон (MLP) архитектуры **2-2-1** с **сигмоидной** функцией активации для решения задачи типа **«исключающее ИЛИ»** (XOR), в которой входные и выходные значения представлены не бинарными 0 и 1, а произвольными вещественными (целыми) числами из диапазона от -10 до 10 , соответствующими логическим классам 0 и 1. Провести сравнение эффективности обучения и качества решения задачи многослойным персептроном в сопоставлении с **однослойным персептроном**, показав принципиальную линейную неразделимость задачи XOR.

Задачи лабораторной работы

1. Реализовать многослойный персептрон архитектуры 2-2-1 (2 входных нейрона, 2 нейрона скрытого слоя, 1 выходной нейрон) с сигмоидной функцией активации на всех слоях.
2. Реализовать алгоритм обучения сети методом обратного распространения ошибки (backpropagation) в онлайн-режиме (обновление весов после каждого примера).
3. Обучить сеть на полной таблице истинности (4 примера) своего варианта до достижения заданного критерия остановки по суммарной ошибке $E_s \leq E_e$.
4. Реализовать и обучить однослойный персептрон (без скрытого слоя, один сигмоидный нейрон) на тех же данных и зафиксировать результат его обучения.
5. Сравнить многослойный и однослойный персептроны по следующим критериям:
 - количество эпох до сходимости (либо факт недостижения критерия остановки за заданное максимальное число эпох),
 - итоговая суммарная ошибка, точность (ассигасу) воспроизведения всех 4 примеров таблицы истинности.
6. Реализовать режим функционирования обученной сети: пользователь вводит пару чисел (A, B) из диапазона $[-10; 10]$ (не обязательно совпадающих с c_0 и c_1), сеть выдаёт выходное значение $\hat{y}$ и указывает, к какому классу (c_0 или c_1) оно ближе.
7. Сформулировать вывод по результатам сравнения (см. вопросы ниже).

Вариант 6 ($c_0 = -9, c_1 = 6$)

A	B	$A \oplus B$
-9	-9	-9
-9	6	6
6	-9	6
6	6	-9

Описание архитектуры:

- Многослойный персептрон (MLP 2-2-1): Состоит из 2 входных нейронов (признаки A, B), 2 нейронов скрытого слоя и 1 выходного нейрона. Все нейроны используют сигмоидную функцию активации. Всего 9 обучаемых параметров. Наличие скрытого слоя позволяет сети формировать нелинейную границу решений.
- Однослойный персептрон (SLP): Состоит из 2 входных нейронов и 1 выходного нейрона с сигмоидой. Всего 3 обучаемых параметра. Способен строить только линейную разделяющую поверхность (прямую).
- Общие параметры обучения: инициализация весов случайными малыми числами, метод обратного распространения ошибки в онлайн-режиме, скорость обучения $\eta=0.5$, критерий остановки $E_s \leq 0.01$.

Результаты обучения:

Многослойный персептрон (MLP 2-2-1):

Количество эпох до сходимости: 2102

Итоговая суммарная ошибка (E_s): 0.00999 (критерий $E_s \leq 0.01$ выполнен)

Точность (Ассигасу) на таблице истинности: 100% (4 из 4 примеров классифицированы верно).

Однослойный персептрон (SLP 2-1):

Количество эпох до сходимости: Не достигнута за 10 000 эпох.

Итоговая суммарная ошибка (Es): 0.53195 (обучение застряло в локальном минимуме).

Точность (Ассигасу) на таблице истинности: 25% (верно угадан только 1 пример из 4: [6, 6]).

Код программы:

```
import numpy as np
import matplotlib.pyplot as plt

C0 = -9.0
C1 = 6.0
TARGET_ERROR = 0.01
MAX_EPOCHS = 10000
LEARNING_RATE = 0.5

def sigmoid(x):
    return 1.0 / (1.0 + np.exp(-np.clip(x, -250, 250)))

def sigmoid_derivative(s):
    return s * (1.0 - s)

def normalize(val, c_min=C0, c_max=C1):
    return (val - c_min) / (c_max - c_min)

def denormalize(val, c_min=C0, c_max=C1):
    return val * (c_max - c_min) + c_min

X_raw = np.array([
    [C0, C0],
    [C0, C1],
    [C1, C0],
    [C1, C1]
])
y_raw = np.array([C0, C1, C1, C0]).reshape(-1, 1)

X_norm = normalize(X_raw)
y_norm = normalize(y_raw)

class MLP:
    def __init__(self, lr):
        self.lr = lr

        self.W1 = np.random.randn(2, 2) * 0.5
        self.b1 = np.random.randn(1, 2) * 0.5
        self.W2 = np.random.randn(2, 1) * 0.5
        self.b2 = np.random.randn(1, 1) * 0.5

    def forward(self, x):
        self.z1 = np.dot(x, self.W1) + self.b1
        self.a1 = sigmoid(self.z1)
        self.z2 = np.dot(self.a1, self.W2) + self.b2
        self.a2 = sigmoid(self.z2)
        return self.a2

    def train_step_online(self, x, y_true):

        pred = self.forward(x)

        error = y_true - pred
        delta2 = error * sigmoid_derivative(pred)

        delta1 = np.dot(delta2, self.W2.T) * sigmoid_derivative(self.a1)

        self.W2 += self.lr * np.dot(self.a1.T, delta2)
        self.b2 += self.lr * delta2
        self.W1 += self.lr * np.dot(x.T, delta1)
        self.b1 += self.lr * delta1
```

```

        return 0.5 * np.sum(error ** 2)

def calc_total_error(self, X, y):
    total_err = 0.0
    for i in range(len(X)):
        pred = self.forward(X[i:i + 1])
        total_err += 0.5 * np.sum((y[i:i + 1] - pred) ** 2)
    return total_err

class SLP:
    def __init__(self, lr):
        self.lr = lr
        self.W = np.random.randn(2, 1) * 0.5
        self.b = np.random.randn(1, 1) * 0.5

    def forward(self, x):
        self.z = np.dot(x, self.W) + self.b
        self.a = sigmoid(self.z)
        return self.a

    def train_step_online(self, x, y_true):
        pred = self.forward(x)
        error = y_true - pred
        delta = error * sigmoid_derivative(pred)

        self.W += self.lr * np.dot(x.T, delta)
        self.b += self.lr * delta

        return 0.5 * np.sum(error ** 2)

    def calc_total_error(self, X, y):
        total_err = 0.0
        for i in range(len(X)):
            pred = self.forward(X[i:i + 1])
            total_err += 0.5 * np.sum((y[i:i + 1] - pred) ** 2)
        return total_err

print("Запуск обучения многослойного персептрона (MLP 2-2-1)...")
mlp = MLP(lr=LEARNING_RATE)
mlp_history = []
mlp_converged = False

for epoch in range(MAX_EPOCHS):
    indices = np.arange(len(X_norm))
    np.random.shuffle(indices)

    epoch_error = 0.0
    for i in indices:
        x_sample = X_norm[i:i + 1]
        y_sample = y_norm[i:i + 1]
        epoch_error += mlp.train_step_online(x_sample, y_sample)

    mlp_history.append(epoch_error)

    if epoch_error <= TARGET_ERROR:
        print(f"✓ MLP: Сходимость достигнута на эпохе {epoch + 1}. Итоговая ошибка: {epoch_error:.5f}")
        mlp_converged = True
        break

if not mlp_converged:
    print(f"✗ MLP: Не сошелся за {MAX_EPOCHS} эпох. Итоговая ошибка: {mlp_history[-1]:.5f}")

print("\nЗапуск обучения однослойного персептрона (SLP 2-1)...")
slp = SLP(lr=LEARNING_RATE)
slp_history = []
slp_converged = False

for epoch in range(MAX_EPOCHS):
    indices = np.arange(len(X_norm))
    np.random.shuffle(indices)

    epoch_error = 0.0
    for i in indices:
        x_sample = X_norm[i:i + 1]
        y_sample = y_norm[i:i + 1]

```

```

    epoch_error += slp.train_step_online(x_sample, y_sample)

slp_history.append(epoch_error)

if epoch_error <= TARGET_ERROR:
    print(f"✓ SLP: Сходимость достигнута на эпохе {epoch + 1}. Итоговая ошибка: {epoch_error:.5f}")
    slp_converged = True
    break

if not slp_converged:
    print(f"✗ SLP: Не сошелся за {MAX_EPOCHS} эпох. Итоговая ошибка: {slp_history[-1]:.5f}")

def evaluate_network(model, test_cases, model_name):
    print(f"\n--- Проверка работы: {model_name} ---")
    print(f"{'Вход (A, B)':<15} | {'Выход ŷ':<10} | {'Ближайший класс'}")
    print("-" * 48)

    for a, b in test_cases:
        x_test = normalize(np.array([[a, b]]))
        out_norm = model.forward(x_test)
        out_real = denormalize(out_norm)[0][0]

        dist_c0 = abs(out_real - C0)
        dist_c1 = abs(out_real - C1)
        closest_class = C0 if dist_c0 < dist_c1 else C1

        print(f"{'{a:>3}, {b:>3}'} | {'{out_real:>8.2f}'} | {'{closest_class}'}")

test_inputs = [
    (-9, -9), (-9, 6), (6, -9), (6, 6),
    (0, 0), (-5, 3), (2, -7), (6, 0)
]

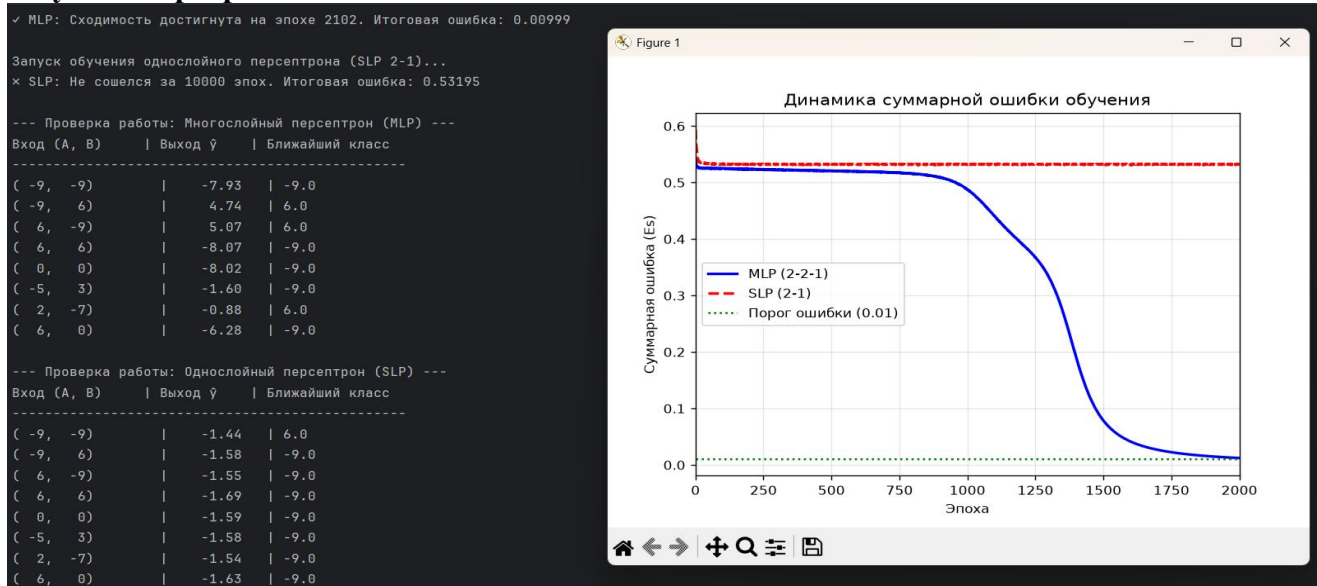
evaluate_network(mlp, test_inputs, "Многослойный перцептрон (MLP)")
evaluate_network(slp, test_inputs, "Однослойный перцептрон (SLP)")

plt.figure(figsize=(10, 5))
plt.plot(mlp_history, label='MLP (2-2-1)', color='blue', linewidth=2)
plt.plot(slp_history, label='SLP (2-1)', color='red', linestyle='dashed', linewidth=2)
plt.axhline(y=TARGET_ERROR, color='green', linestyle=':', linewidth=1.5, label=f'Порог ошибки ({TARGET_ERROR})')

plt.title('Динамика суммарной ошибки обучения (Вариант 6: c0=-9, c1=6)')
plt.xlabel('Эпоха')
plt.ylabel('Суммарная ошибка (Es)')
plt.legend()
plt.grid(True, alpha=0.3)
plt.xlim(0, min(2000, MAX_EPOCHS))
plt.show()

```

Результат программы:



Результаты работы обученной сети в режиме функционирования:

Обученные сети протестированы на 4 примерах таблицы истинности и 4 произвольных парах чисел. Выходное значение $\hat{y}$ денормализуется и сравнивается с классами $c_0 = -9$ и $c_1 = 6$.

Вход (A, B)	Ожидаемый класс	MLP выход ($\hat{y}$)	MLP класс	SLP выход ($\hat{y}$)	SLP класс
(-9, -9)	-9	-7.93	-9	-1.44	6
(-9, 6)	6	4.74	6	-1.58	-9
(6, -9)	6	5.07	6	-1.55	-9
(6, 6)	-9	-8.07	-9	-1.69	-9
(0, 0)	—	-8.02	-9	-1.59	-9
(-5, 3)	—	-1.60	-9	-1.58	-9
(2, -7)	—	-0.88	6	-1.54	-9
(6, 0)	—	-6.28	-9	-1.63	-9

Многослойный персептрон верно классифицирует все примеры из таблицы истинности (ассигура 100%) и корректно обобщает закономерность на новые данные. Однослойный персептрон выдает значения в узком диапазоне (около -1.5) для любых входов, не сумев разделить классы XOR (ассигура 25%).

Вывод: в ходе работы реализованы однослойный (SLP) и многослойный (MLP 2-2-1) персептроны для задачи XOR с числовыми классами $c_0 = -9$ и $c_1 = 6$.

1. Сможет ли однослойный персептрон решить задачу XOR?

Нет, не смог. За 10 000 эпох критерий остановки не достигнут (ошибка $E_s \approx 0.53$). Точки классов XOR расположены по диагонали и линейно неразделимы одной прямой — фундаментальное ограничение однослойных сетей.

2. Сможет ли многослойный персептрон 2-2-1 корректно решить задачу?

Да, решил корректно. Сходимость достигнута за 2102 эпохи с итоговой ошибкой $E_s = 0.00999$ (критерий ≤ 0.01 выполнен) и точностью 100% на таблице истинности.

3. Роль скрытого слоя. Он необходим для построения нелинейной разделяющей поверхности. Нейроны скрытого слоя создают несколько линейных границ, которые выходной нейрон нелинейно комбинирует, что позволяет разделить классы XOR.

4. Влияние числового диапазона. Поскольку сигмоида выдаёт значения строго в интервале $(0, 1)$, прямое обучение на числах -9 и 6 невозможно. Потребовалась нормализация данных в $[0, 1]$, что предотвратило насыщение функции активации и обеспечило стабильный градиентный спуск.

5. Сравнение с классическим XOR. Математическая сложность задачи не изменилась. Нормализация является линейным преобразованием, которое не меняет геометрическое расположение точек. Задача остаётся столь же линейно неразделимой, как и в классическом случае с 0 и 1 .

В итоге, экспериментально подтверждено, что многослойный персептрон с хотя бы одним скрытым слоем принципиально превосходит однослойный персептрон при решении нелинейно разделимых задач, таких как XOR. Замена бинарных классов на произвольные числовые значения не меняет фундаментальной сложности задачи, но требует корректной предобработки данных через нормализацию.